

モダンSaaS開発 & プロジェクト推進 総合プレイブック

GitHub × Vercel × Supabase を用いた高速開発・アーキテクチャ・やること管理
ノウハウ

分類: 汎用Webサービス・SaaS開発標準
ガイド

更新日: 2026年
9月

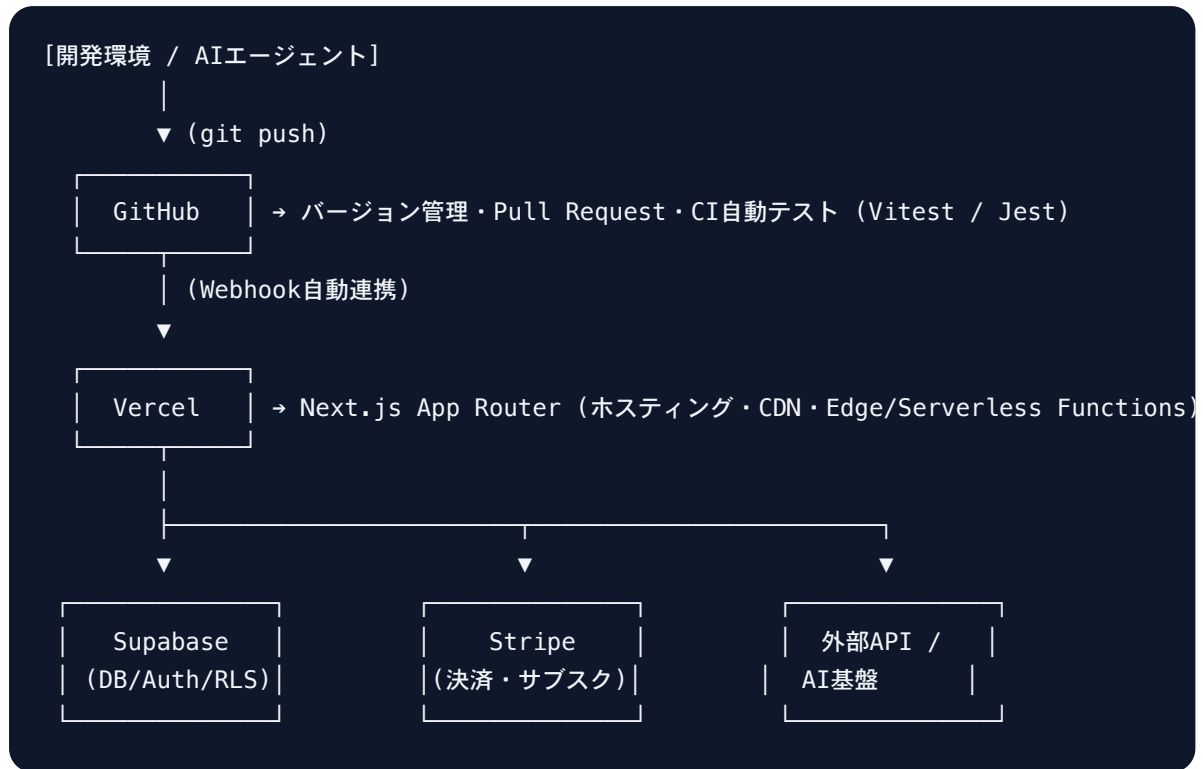
推奨環境: Next.js / TypeScript /
PostgreSQL

【本ドキュメントの目的】

本ドキュメントは、Next.js、GitHub、Vercel、Supabase、および外部決済・AI APIを組み合わせ
てWebサービス（SaaS）を迅速かつ安全に立ち上げ・スケールさせるための**システム設計アーキテクチャ**
および**プロジェクト推進（やること管理・意思決定）のノウハウ**を体系化した汎用プレイブックです。
特定のサービス名やドメインに依存せず、新規事業の立ち上げ時に標準テンプレートとして活用できま
す。

1 システム構造 & インフラアーキテクチャ

1. 全体パイプライン (GitHub → Vercel → Supabase)



⚡ インフラ保守ゼロ

サーバーレスによりOSパッチやスケーリング監視が不要。PVがゼロの時は費用も極小。

🚀 デプロイの自動化

GitHubへpushするだけでVercelが自動ビルド。PRごとのプレビューURLで即時レビュー可能。

🗄️ フルスタックDB

SupabaseによりPostgreSQL、認証、RLS、ファイル保管、リアルタイム連携を即座に利用。

2. オフラインファースト & クラウド二重化設計

B2Cや習慣化SaaSでは、「ネット環境が不安定でも快適に動く」「登録前にすぐ試せる」ことがCVR（登録率）と継続率を大きく左右します。

- ① **ゲスト体験:** 登録不要で即時利用可能にし、LocalStorage/IndexedDBへ一時保存（超高速・離脱防止）。
- ② **クラウド自動移行:** 会員登録・ログインが完了した瞬間に、端末内の利用履歴をSupabaseへ一括自動アップロード。
- ③ **複数端末同期:** PCやスマホのログイン時にクラウドから最新データを自動ダウンロードし、差分をマージ。

2 セキュリティ & データ整合性の鉄則

1. Row Level Security (RLS) の徹底

Supabaseではフロントエンドから直接DB操作が可能です、**全テーブルでRLSを有効化しないと他人のデータが丸見えになります。**

-- 本人のみ読み書きを許可するポリシー例

```
ALTER TABLE public.user_data ENABLE ROW LEVEL SECURITY;  
CREATE POLICY "User owns data" ON public.user_data FOR ALL USING (auth.uid() =  
user_id);
```

※ 管理者集計や定期バッチは、サーバーサイドAPIからのみ SUPABASE_SERVICE_ROLE_KEY を使用して実行する。

2. 決済情報の完全非保持化 (PCI-DSS準拠)

自社サーバーにクレジットカード番号を保持・通過させることはセキュリティ・法務上絶対に避けるべきです。

- **Stripe Checkout / Customer Portal** の全面採用により、カード情報はStripe内で完結。
- 自社DBには「Stripe顧客ID (cus_xxx)」のみを紐付け、カード番号は一切持たない。
- Webhookイベント（課金成功・解約・期限切れ）を受信してSupabaseのプランステータスを自動更新。

3. サーバーレス環境での状態管理（インメモリ禁止）

VercelのServerless Functionsはアクセスごとに異なるコンテナが起動し、数分アクセスがないと破棄されます。「**メモリ上の変数（Mapなど）に保存したデータは再起動・デプロイで必ず消える**」ため、アクセスログや状態値は必ずSupabaseやRedisなどの外部DBへ永続化する。

3 プロジェクト推進 & 「やること管理」ノウハウ

1. 3層マネジメント分業体制

① オーナー / PO

最高意思決定
事業方針・最終承認

② 統括担当 / PM

要件定義・仕様書作成
試算・品質監査

③ 実務担当 / エンジニア

コード実装・単体テスト
ビルド検証・デプロイ

【最重要鉄則】「勝手に直さない」意思決定プロトコル
開発中に問題や不整合を発見しても、実装者は勝手にコードを書き換えてはならない。

- 検知:** どこに何の懸念・問題があるかを特定。
- 提案:** 修正案・選択肢を整理して提示。
- 判断:** オーナーの確認・承認を得る。
- 着手:** 承認後に初めて実装を行い、テスト確認を経て完了報告する。
これにより、「意図と異なる改修」や「仕様の先祖返り」を完全に防止できる。

2. ドキュメント体系の標準テンプレート

ファイル名	役割・記録内容	更新頻度
PROGRESS_MEMO.md	全体進捗、決定事項ログ、直近ToDoの一元管理	決定事項発生ごと
FEEDBACK_LOG.md	顧客・テストの声（VoC）と対応ステータス管理簿	FB受領ごと
weeklylog.md	週次での決定事項・実装・直面した課題・解決策の定点記録	毎週月曜朝
SYSTEM_SPECIFICATION.md	サービスの基本仕様・価格・DB・API原価・ルールのマスター設計書	仕様変更時（即時）
XXXX_SPEC.md	機能単位の詳細実装指示書（背景、要件、テスト方針）	機能着手ごと

4 事業採算・コスト管理 & リスク自動計算ノウハウ

1. API従量課金SaaSの原価シミュレーション

AIやSMSなどの従量課金APIを組み込む場合、概算ではなく「1リクエストあたりの実測原価」を厳密に計算します。

実測ベースの単価算出:

トークン数、録音秒数、文字数を実測し、1ドル150円等の為替レートで1回あたりの原価を算出（例: 1回約0.32円）。

キャッシュ再利用による圧縮:

同一入力の再試行時は生成済みオブジェクトをメモリキャッシュから再利用し、2回目以降のコストを0円にする。

2. 顧客ごとの損益分岐点（採算限界）自動計算式

有料プランを「使い放題」として提供する場合、管理画面で各顧客の利用ペースと赤字化リスクを自動計算させます。

月間損益分岐回数 = (月額料金 - Stripe手数料 3.6%) ÷ 1回あたり原価

1日あたり限界回数 = 月間損益分岐回数 ÷ 30日



安全 (Safe)

限界の50%未満利用



注意 (Warning)

限界の50%～80%利用



警戒 (Danger)

限界の80%以上利用

5 新規プロジェクト立ち上げ時のチェックリスト

Stage 0: MVP構築 & コア検証

- ☑ Next.js + Tailwind フロントエンド構築
- ☑ コアドメイン機能（提供価値）の動的検証
- ☑ ゲスト体験（LocalStorage一時保存）
- ☑ 単体テスト環境（Vitest / Jest）

Stage 1: 課金・認証・法務

- ☑ Supabase Auth (Email / Google) 導入
- ☑ DBスキーマ設計 & RLSポリシー適用
- ☑ Stripe Checkout / Webhook 決済基盤
- ☑ 法務3画面 (/terms, /privacy, /tokushoho)

Stage 2: クローズドβ & 定着検証

- ☑ 限定テスター30〜50名の招待
- ☑ 不具合報告・問い合わせ窓口設置
- ☑ アクセス解析基盤（Vercel Analytics等）
- ☑ 習慣化ループ（ストリーク・通知）

Stage 3: ローンチ & 収益化始動

- ☑ LP最適化 & SEOメタタグ整備
- ☑ SNS・コミュニティでのパブリック公開
- ☑ 顧客管理（Admin）による採算リスク監視
- ☑ 週次振り返りログによる高速改善